Higgs API Feedback

Issues we hit building a real-time voice booking assistant on Higgs Realtime, STT 3.1 and TTS 3, how we worked around each one, and what would help other builders.

Project: Code Blue, a voice receptionist for clinics (Maple Clinic mock) with emergency pre-emption · **APIs:** Higgs Realtime (`higgs-realtime`), STT 3.1 inside Realtime (`higgs-stt-3.1`), TTS 3 (`higgs-tts-3`), Avatar · **Client:** browser WebSocket with ephemeral client secrets, PCM16 24 kHz, `server_vad` · **Code:** github.com/Shubh8434/hackathon-higgs (tag `rescheduling_appointment`) · **Date:** October 24, 2026

Summary

#	Issue	Impact	Our fix
1	With default `server_vad` , the response starts **before** the input transcript is delivered	Critical	`create_response: false` ; the app processes the transcript, updates instructions, then sends `response.create`
2	Model ignores explicit "never" rules (address follow-ups, inventing availability, holds, ID mismatches, payment steps)	High	App-side guards detect the violation and make the model correct itself; authoritative "NEXT LINE"
3	Bare `response.create` rejected with "No user input" after the assistant spoke last	Medium	Insert a short text user item (`[Clinic screen] …`) before `response.create`
4	No `session.updated` acknowledgement; an invalid session config closes the socket	Medium	Send the full, exact session shape every time; verified settings with a live probe
5	STT 3.1: spoken numbers stay as words, phone numbers and sentences split across turns, domain words misheard, filler words kept	High	Client-side normalization, joining digits across turns, and trusting the model's read-back
6	With `create_response: false` , a reply cancelled by barge-in (or by noise/echo) leaves the conversation stalled	High	Per-turn reply tracking with retries, a watchdog, and re-answering the last turn after a cancel

---

1. The reply starts before the transcript arrives Critical

WHAT HAPPENED

Our app keeps the clinic calendar as the source of truth: when the patient speaks, it parses the request, checks real availability and re-sends the model's `instructions` with the correct answer. Ava (the assistant) nevertheless kept answering with the *previous* turn's information and then invented reasons to justify it. Real transcript (conversation `CONV-20261024-441C67`):

> AVA: The nearest open times are 10:30 AM on Monday, 11:30 AM on Monday or 2:30 PM on Monday. Which works for you?
>
> PATIENT: I think we can go for Wednesday.
>
> AVA: I'm sorry, but Dr. Patel is actually closed on Wednesdays. The nearest open times are 10:30 AM on Monday, 11:30 AM on Monday, or 2:30 PM on Monday.
>
> PATIENT: So, Doctor Patel is available on 11:30 AM on Friday.
>
> AVA: Yes, 11:30 AM on Friday is available. I'm holding that time for you now.

In fact Wednesday had 8 open slots and 11:30 AM Friday was taken. At that turn the app had already computed the correct line ("*On Wednesday, the open times are 9:00 AM, 9:30 AM or 10:00 AM*"), but it arrived too late. The

payment screen and SMS receipt then showed a different time from the one Ava had promised.

EVIDENCE: MEASURED EVENT TIMELINE

We streamed the same TTS-generated utterance ("*I would like to book an appointment on Wednesday please*") into two fresh sessions. Times are from socket open:

Event	Default `server_vad`	`create_response: false`
`input_audio_buffer.speech_stopped`	4,028 ms	4,027 ms
`response.created`	4,031 ms (auto)	none until the app asks
`conversation.item.input_audio_transcription.completed`	4,317 ms (286 ms *after* the reply began)	4,388 ms
App sends `response.create` after processing the transcript	n/a	6,889 ms (test delay)
`response.created` → first output item	4,031 → 4,613 ms	6,917 → 7,491 ms

With the default behaviour, the client **cannot** update instructions based on what the user said before the model starts answering, because the transcript hasn't been delivered yet. Any app that grounds the model in live data (calendars, inventory, account state) is affected.

OUR FIX

Turn off automatic responses and let the app drive each turn. Higgs accepted these OpenAI-compatible fields:

```
turn_detection: {
  type: 'server_vad', threshold: 0.55, prefix_padding_ms: 300,
  silence_duration_ms: 500, min_speech_duration: 0.125,
  create_response: false,      // the app decides when Ava speaks
  interrupt_response: true,    // barge-in still cancels her audio
}

// on conversation.item.input_audio_transcription.completed:
orchestrator.handleUser(transcript);                      // parse + check calendar
ws.send({ type: 'session.update', session: { ...config, instructions } });
ws.send({ type: 'response.create' });                     // now reply with fresh state
```

Re-running the same conversation against the live model afterwards, every answer matched the calendar, and the payment screen and SMS showed the time Ava had said:

> PATIENT: I think we can go for Wednesday.
> AVA: On Wednesday, the open times are 9:00 AM, 9:30 AM or 10:00 AM. Which works for you?
> PATIENT: Is Dr. Patel available at 11:30 a.m. on Friday?
> AVA: Sorry, 11:30 AM on Friday isn't available with Dr. Patel. On Friday, the open times are 9:30 AM, 10:00 AM or 10:30 AM.

We also added a fallback: if no transcript arrives within 5 s of `speech_stopped`, the app sends `response.create` anyway, so Ava never goes silent.

Suggestions for Higgs:

- Document `create_response` / `interrupt_response` for Higgs Realtime. We only found them because the API mirrors OpenAI's.
- Offer a mode that waits for `input_audio_transcription.completed` before auto-creating the response, or send the transcript before `response.created`, so clients can react without managing turns themselves.
- First-class function/tool calling, so the model can read live data at answer time instead of relying on instructions the client re-sends.

---

2. The model ignores explicit "never" rules High

WHAT HAPPENED

Even with up-to-date instructions, the model sometimes broke rules that are critical in a booking flow:

Rule in the instructions	What the model said
"Accept the address exactly as said. NEVER ask for a city, province, postal code, street name…"	"Thanks. Could you please tell me the city and province for that address?" "Thanks. What's the street name for that address?"
"Offer up to three open times. Never list more than three in one turn."	Read out all 10 of Tuesday's times in one sentence.
"Never say you are holding a time unless the NEXT LINE says so."	"Great, your appointment is confirmed for 11:30 AM to 12:00 PM on Friday, November 6th", which was not the held slot (Thursday 3:30 PM).
NEXT LINE: "Which day or time would you like instead?" (the booking had already been found from the spoken ID "PVAASH2")	"The system shows your booking ID as PVA-ASH2, but the one you gave me, PVAASH2, isn't matching. Could you double-check the ID?" It repeated this when the patient said the ID again (conversation `CONV-20261024-ED5173`).
"Do the final check only after name, address and phone are collected." The patient had given only 6 digits.	Read back "your phone number is 921345", then said "The payment screen is now open on your device. Please tap Done", although the app had not opened it.
NEXT LINE: "Done. Your appointment is now 9:00 AM to 9:30 AM on Monday…" (a reschedule; details already on file)	Said the line, then added "To finish updating your record, what is your full name?", applying the new-booking rule to a reschedule.

OUR FIX

We stopped relying on the prompt alone and check every assistant turn (`response.done` transcript) in code:

- **Address follow-up guard:** if Ava asks about city, province, street name, unit or "full address" after an address was given, the app makes her immediately say "*Actually, the address you gave is all I need. And what's the best phone number to reach you?*"
- **Hold guard:** if Ava names a time that differs from the screen, the app moves the hold when that time is open, or makes her correct herself when it isn't: "*Sorry, I need to correct myself: 11:30 AM on Friday is not available… Your appointment is still 10:30 AM to 11:00 AM on Friday.*" While the payment screen is open, it restates the real time.
- **Authoritative next line:** the app's computed reply goes into the instructions as "*NEXT LINE: say this now. Keep every doctor, day, date and time exactly as written.*"
- **State the facts the model tends to doubt:** "The booking is FOUND and VERIFIED: PVA-ASH2 (spoken "P V A, A S H 2"); dashes and spaces don't matter. Never question the booking ID again." and "This booking is complete: don't ask for any details or payment again."

- **Premature-step guard:** if Ava starts the final check or mentions the payment screen while a detail is missing, she is corrected: "*Sorry, I'm not quite done. I only have 6 digits of your phone number…*"

```
// on response.done
if (isAddressFollowUp(text)) nudge(`Actually, the address you gave is all I need.
${nextQuestion}`);
if (isHoldAnnouncement(text)) checkAnnouncedHold(text);  // sync the hold or correct Ava
```

Suggestions for Higgs:

- Stronger adherence to explicit negative constraints ("never ask X") in the system instructions.
- Per-response instructions on `response.create` (e.g. `response.instructions`), so the next line can be pinned for one turn without rewriting the whole session.
- Structured output or tool calls for actions like "hold slot X", so the app executes the action instead of parsing it from free speech.

---

3. `response.create` rejected with "No user input" Medium

WHAT HAPPENED

Some events happen on screen, not in speech: the patient taps **Done** on the payment screen, the safety net detects an emergency, or a page refresh resumes a session. We need Ava to speak right away. When the assistant had spoken last, a plain `response.create` was rejected with the error "*No user input*".

OUR FIX

```
ws.send({ type: 'conversation.item.create', item: { type: 'message', role: 'user',
  content: [{ type: 'input_text', text: '[Clinic screen] The patient tapped Done and the
payment went through. Tell the patient: "…"' }] } });
ws.send({ type: 'response.create' });
```

This works, but the synthetic "user" message ends up in the conversation history, where the model can later treat it as something the patient said.

Suggestions for Higgs:

- Allow assistant-initiated responses (as the OpenAI Realtime API does), or accept a `system`-role item for app events.
- Document this rule in the Realtime reference.

---

4. No `session.updated`, and invalid config closes the socket Medium

WHAT HAPPENED

We re-send `session.update` on every state change to keep instructions current. In our probes, Higgs sent `session.created` but never `session.updated`, so the client can't confirm an update (or a new field like `create_response`) was applied. We confirmed `create_response: false` only indirectly, by observing that no automatic response was created. Separately, a session object with an unexpected shape closed the socket instead of returning an error event, which made early debugging slow.

OUR FIX

We send the complete, identical session shape every time (only `instructions` changes) and verify behaviour with a scripted probe: TTS audio streamed into a test session, with the event timeline logged.

Suggestions for Higgs:

- Emit `session.updated` with the effective configuration after each `session.update`.
- Return an `error` event naming the invalid or unknown field instead of closing the connection.

---

5. STT 3.1 on names, addresses and phone numbers High

WHAT HAPPENED

Collecting patient details by voice exposed several transcription patterns:

Patient said	STT 3.1 delivered	Problem
"six four seven, nine two zero, four five four eight"	`Six four seven nine two zero four five four eight.` or several separate turns	Digits stay as words; pauses between digit groups end the turn, so one phone number arrives as 2–3 user turns (one fragment: `A five, five foot.`)
"123 Street East"	`Straightest one, two, three.` / `Straight east` / `Status 123`	Street names misheard; house numbers as words
Name, after "What's your name?"	`Ah Subham.` / `Myself Subham Sharma.`	Filler and lead-in words kept as part of the name
A phone number given early	`Eight four three`	With no digit formatting, number words were captured as a *name*
"Can you please reschedule the appointment for Monday?"	Two finals: `Can you please residual the?` then `Can you please reschedule the appointment for Monday?`	One utterance split into two turns, the first with the key word misheard ("residual")
"I think for me, 3 p.m. would be a good time." then "Hello."	`I think for me, 3 p.m. would be good time.` then `I think for me, 3 p.m. would be good time.Hello.`	A later transcript repeated the earlier sentence, so the same request arrived twice
A booking ID, "P V A, A S H 2"	`PVAASH2`	Letters merged without the dash, so it no longer matched `PVA-ASH2` textually (and the model then disputed it, see issue 2)

OUR FIX

- Normalize digit words to digits (including "for" → 4, "to" → 2) and **join digits across turns** until a full 10-digit number is reached.
- Strip lead-ins and fillers from names and addresses ("ah", "myself", "my address is", "I think it's").
- Parse booking IDs from letter-by-letter speech; treat a repeated, already-held time as no change; require exactly 10 phone digits before accepting a number.
- Prefer the **model's read-back** ("your name is …, your address is …, your phone number is 647-920-4548"), which is often cleaner than the raw transcript, then confirm with the patient ("Is that all correct?").

```
phoneBuffer += phoneDigits("four six two,");   // "843" + "462" + "8540" → (843) 462-8540
cleanAddress("I think it's Street East. One, two, three")  // → "Street East 123"
extractBookingId("p v a nine six five w")                  // → "PVA-965W"
```

Suggestions for Higgs:

- An inverse-text-normalization option for STT (numbers, phone numbers, addresses, times written as digits).
- Keyword/phrase boosting per session (street names, doctor names like "Dr. Patel", "Rahman").
- Smarter end-of-turn detection while a digit sequence is being spoken (semantic VAD, or a longer silence window mid-number).

---

6. A cancelled reply stalls the conversation High

WHAT HAPPENED

With `create_response: false` (our fix for issue 1) and `interrupt_response: true`, Higgs cancels Ava's reply as soon as VAD detects speech. When that "speech" is a real sentence, the app answers its transcript. When it is noise, or Ava's own voice echoing from the speakers, no usable transcript follows. Nothing triggers a new response, so **Ava goes silent** and the patient sees an empty "…" bubble. In one session the patient said "*I think for me, 3 p.m. would be good time.*", the slot was held on screen, and Ava never answered; the last bubble was an empty "…".

EVIDENCE: MEASURED BARGE-IN TIMELINE

We streamed a TTS sentence, asked for a reply, then streamed "Hello." 0.7 s into Ava's answer (times from socket open):

Time	Event
1,571 ms	transcription completed: "I think 3 p.m. would be a good time for me." → app sends `response.create`
1,597 ms	`response.created`
2,298 ms	patient starts speaking ("Hello.")
2,409 ms	`response.output_audio_transcript.done`: "That sounds like a great time for you, and I can certainly check if we have any availability at 3 PM." (the *full* sentence, though almost none of it was played)
2,415 ms	`response.done` with `status: "cancelled"`, then `input_audio_buffer.speech_started`
2,777 ms	transcription completed: "Hello." → app sends `response.create`; reply completes normally

Two gaps: the cancellation carries no reason (barge-in vs. noise), and the cancelled reply's transcript doesn't say how much the patient actually heard.

OUR FIX

- **Per-turn reply tracking:** every patient sentence gets a turn number; the app retries `response.create` about every second while Ava is busy, and a **watchdog** asks again if `response.created` doesn't arrive within 4 s.
- **Recover after a cancel:** if no new sentence arrives within 2.5 s of a cancelled reply, Ava answers the last turn again.
- Treat a response that has been silent for 12 s without `response.done` as finished; drop empty "…" bubbles; keep cut-off replies in the history, marked "cut off".
- Recommend headphones for demos to avoid echo-triggered barge-ins.

```
onResponseDone(text, status) {
  if (status === 'cancelled') return recoverAfterCancel(turnSeq);   // re-answer if nobody
spoke
}
requestReply(turn) → rt.reply() || setTimeout(() => requestReply(turn), 1000)
```

Suggestions for Higgs:

- Include a cancel reason on `response.done` (e.g. `turn_detected` vs. `client_cancelled`), and how much audio was played (`audio_end_ms`) or a truncated transcript.
- An option to interrupt only on confirmed speech (non-empty transcript or a minimum duration), so noise and echo don't cancel replies.
- When a turn produces an empty transcript, emit an explicit event so clients can resume the previous reply.

What worked well

- **Ephemeral client secrets** kept the API key server-side with almost no code.
- **Barge-in**: `speech_started` arrives quickly, so stopping playback on it feels natural.
- **TTS 3 with `response_format: "pcm"`** made it easy to stream synthetic patient speech into a Realtime session for demos and automated end-to-end tests.
- **Latency**: about 0.6 s from `response.create` to the first audio item in our tests, fine for natural turn-taking.
- **OpenAI Realtime compatibility** meant existing knowledge and event names carried over. It's also how we found `create_response`.

Code Blue · Higgs API feedback · All examples are taken from real sessions recorded by the app (conversation IDs shown) and from scripted probes against the live Higgs Realtime API.